Course Code: FM102

Course Title: Coding Café

Course intended for: Freshmore

Course Credits: 1

Class Schedule: 15 weeks; one session per week (lecture or lab or a combination)

Lead Instructor: Raghav Awasty

Office No.: A1-WS429

Email ID: raghav.awasty@plaksha.edu.in

Teaching Fellows: Ritvik Ranjan (ritvik.ranjan.tlf24@alumni.plaksha.edu.in), Pawan Saini (pawan.saini@plaksha.edu.in)

Course Description

Coding Café is a hands-on introductory course in Python programming, designed for freshmore students. The course builds foundational programming skills through small, practical coding tasks with minimal theoretical overhead, and introduces the developer tools (the terminal, Git, and GitHub) that programmers are expected to know and use professionally.

The course emphasizes experimentation, play, and iteration over memorization. Students are encouraged to explore and "fail fast" during labs; clarity of thinking is more important than syntax perfection. Through regular dev-tool practice, good developer habits are reinforced weekly rather than taught once.

Learning Objectives

1. Understand the basic syntax and semantics of Python, and relate Python constructs to their C counterparts.
2. Apply control structures (conditionals, loops) to solve computational problems.
3. Write modular, readable programs using functions and modules.
4. Use strings, lists, dictionaries, and files effectively in programs.
5. Apply object-oriented programming concepts (classes, objects, inheritance) in Python.
6. Use the terminal, pip, Git, and GitHub as part of a basic developer workflow.

Learning Outcomes

After completing this course, students should be able to:

1. Write Python programs using variables, conditionals, loops, functions, and core data structures.
2. Explain and apply object-oriented programming concepts in Python.
3. Relate Python constructs to their C counterparts encountered in FM111.

4. Use the command line to navigate the file system, run programs, and manage packages.
5. Use Git and GitHub to version, share, and submit code.
6. Read error messages, debug systematically, and handle exceptions.
7. Read from and write to files, and use Python libraries to extend program logic.

Recommended Textbooks and Online Resources

- John Guttag – Introduction to Computation and Programming Using Python: With Application to Understanding Data (MIT Press)
- Al Sweigart – Automate the Boring Stuff with Python (free online)
- Python Docs (official)
- Dev Tools course – https://dev-tools.mo8it.com/ (terminal and Git reference)
- The Missing Semester – https://missing.csail.mit.edu (dev-tools and coding practices)
- Pro Git, Chapters 1–2 (free online)
- W3Schools Python Tutorial
- Reddit r/learnpython

Assessments and Grading: [Grading will be relative]

- Weekly Lab Work: 40% – graded lab submissions (via Git from Week 6 onward)
- Quizzes: 20% [4 × 5] – short quizzes at concept milestones
- In-Person Assignments: 20% [2 × 10] – proctored assignments held in lab (Weeks 9 and 14)
- Final Programming Exam: 20% – Date: TBA
- [Optional] Bonus credit: up to 10% – possible coding challenges across the weeks (may or may not be implemented)

Weekly Class Plan (subject to minor changes based on the pace of the course)

Week	Session	Topics	Assessment
Week 1	Lab	Environments & First Programs: installing Python, VS Code, running .py scripts, print/input, first programs	
Week 2	Lecture + Lab	Dev-Tools I — The Terminal: navigation, paths, files & directories, running Python from the shell, pip, tab-completion & history	
Week 3	Lab	Variables, Data Types & Operators: dynamic typing (vs. C declarations), arithmetic/comparison/logical operators, I/O, f-strings	

Week 4	Lab	Conditionals & Branching: if/elif/else, Boolean logic, indentation-as-syntax; decision-making problems	Quiz 1
Week 5	Lab	Loops & Iteration: for-in, while, range, loop patterns (accumulate, search, count); small games; reading your first tracebacks	
Week 6	Lecture + Lab	Dev-Tools II — Git & GitHub: version control, init/add/commit, push, writing a README; all subsequent lab work submitted via Git	
Week 7	Lab	Functions & Modules: def, arguments & defaults, return values, scope, imports; code quality — naming, docstrings, refactoring	Quiz 2
Week 8	Lab	Strings & Lists: slicing, methods, comprehensions; text-processing tasks (word counts, ciphers)	
Week 9	Assessment	In-Person Assignment 1 (proctored, in lab): variables, conditionals, loops, functions, strings & lists	Assignment 1 (10%)
Week 10	Lab	Dictionaries, Sets & File Handling: reading/writing text and CSV files, try/except for robust file code; building a small lookup tool	Quiz 3
Week 11	Lecture + Lab	Object-Oriented Programming I: why objects, defining classes, attributes, methods, __init__, creating instances	
Week 12	Lab	Object-Oriented Programming II: inheritance, method overriding, a taste of polymorphism; modelling a small system	
Week 13	Lab	Libraries & Automation: random, requests, intro matplotlib; pathlib/os batch file tasks — automating the boring stuff	Quiz 4
Week 14	Assessment	In-Person Assignment 2 (proctored, in lab): OOP, dictionaries, file handling, libraries	Assignment 2 (10%)
Week 15	Lecture	Revision & Wrap-Up: course review, exam preparation, bonus challenge showcase	

Makeup Policy

If a student misses any evaluation component due to an extenuating circumstance or an authorized official engagement, there exists a provision to apply for makeup. Grounds that qualify for applying for makeup include serious medical emergency or health issue, personal/family emergencies or bereavement, and representing the University as part of its official contingent. The student should submit a makeup application to the instructor of record and provide all necessary documentation in

support of the case. The instructor of record will decide whether or not to award the makeup. For those specific cases that are recommended by the Student Life team/OAA for excused absence, makeup will be awarded.

Attendance Policy

Excused absences with valid documentation (medical, academic duty) will not penalize attendance scores (wherever applicable).

Collaborative Work

Pair programming is encouraged during designated labs. All in-person assignments, quizzes, and the final exam must be individual efforts unless explicitly stated otherwise.

Scholastic Misconduct

It is expected that all students follow the highest standard of academic practice when participating in any evaluation component. Having a zero–tolerance for academic dishonesty, any case of misconduct, however minor, will be dealt with appropriately, and may be reported to the Academic Integrity Committee for necessary administrative action.